THESIS LEARNING & RESEARCH SUMMARY

# Filtered Vector Search on Proximity Graphs

Current preparation status and learning progress

| | |
|---|---|
| Student | Shafikul Islam Marwan |
| Thesis direction | Vector databases / filtered vector search / proximity-graph search |
| Current study source | CMU 15-445/645 Intro to Database Systems, Fall 2025 |
| Completed in depth | Lectures #03 Database Storage I and #04 Memory Management & Buffer Pools; Lecture #01 completed as an introductory foundation |
| Current status | Foundational database concepts completed; main vector-search algorithms are being investigated but are not yet deeply studied or implemented |

## 1. Executive Summary

**Purpose.** This document summarizes what I have learned so far in preparation for an undergraduate thesis on filtered vector search over proximity graphs. The current work is intentionally foundation-first: I have first studied how a database stores, retrieves, and moves data between persistent storage and main memory before moving into vector indexes and filtered ANN algorithms.

**Current research position.** I am treating the topic as a research direction rather than claiming that a final algorithm has already been decided. The initial plan is to understand the database and vector-search foundations, study existing filtered-search methods, reproduce representative baselines, identify a concrete bottleneck, and only then decide the exact optimization to investigate.

**Current level of confidence:** I can explain the basic database storage and memory-management concepts below. I have an initial conceptual understanding of ANN/HNSW/filtered vector search and have surveyed relevant research resources, but I am not presenting those advanced topics as mastered yet.

## 2. What I Have Learned So Far

### 2.1 CMU Lecture #01: Relational Model & Algebra

- Learned the role of the relational model and relational algebra as the logical foundation of database systems.
- Understood that a database course moves from the logical view of data toward the physical/software mechanisms that store and execute queries.

Note: Lecture #01 is treated here as introductory foundation; the detailed study notes currently maintained are for Lectures #03 and #04.

### 2.2 CMU Lecture #03: Database Storage I — Files, Pages, Tuples

This is the first lecture I studied in depth for the physical database layer. The main concepts I can currently explain are:

- Database system layers: disk/storage management, buffer management, access methods, and the higher-level query processing stack.
- Database files on non-volatile storage and the idea that a database internally organizes its data into fixed-size pages.
- Page directories as metadata structures used to locate and manage pages and their offsets/free-space information.
- Database page size and the distinction between database-level pages and lower-level storage/OS concepts.
- Slotted pages: page header, slot array, tuple area, and free-space region.
- Tuple/record representation as bytes, including tuple headers, null information, transaction visibility metadata, and attribute data.
- Record IDs / tuple IDs as logical identifiers that can encode a physical location such as file, page, and slot.
- Variable-length attributes and the need to handle large values through overflow/out-of-page storage mechanisms.

- Why indirection through a slot array allows tuples to move inside a page during compaction without forcing every external reference to use a raw byte offset.
- Basic motivation for alignment/padding and why physical representation matters for efficient execution.

### 2.3 CMU Lecture #04: Memory Management & Buffer Pools

I then studied how the DBMS moves database pages between persistent storage and main memory. Key concepts understood:

- Buffer pool: an area of main memory used to cache database pages needed by running operations.
- Frames: fixed-size memory slots that hold pages in the buffer pool.
- Page table: in-memory metadata that tracks which page is currently in which frame.
- Buffer hit vs. buffer miss: whether the requested page is already resident in memory or must be read from storage.
- Pinning/unpinning: preventing a page from being evicted while an operation is still using it.
- Dirty pages: pages modified in memory that eventually need to be written back to persistent storage.
- Replacement policies such as LRU, Clock, LRU-K, and ARC, and the idea that page eviction must balance recency/frequency against implementation overhead.
- Latch vs. lock at a conceptual level: latches protect DBMS internal structures, while locks are used for higher-level transaction/concurrency semantics.
- Why DBMSs need deliberate control over memory and I/O behavior rather than treating the operating system page cache as the complete storage-management solution.
- The connection between memory-management policy and performance: unnecessary page movement and poor locality can increase latency.

## 3. Concepts I Can Now Connect Across the Two Lectures

The most important outcome of these lectures is that I can now see the physical path followed by a database operation:

| Layer | What it means |
|---|---|
| Persistent storage | Database files survive power loss and contain pages. |
| Page | Fixed-size unit in which the database organizes data and metadata. |
| Buffer pool | Main-memory cache of pages currently needed by the DBMS. |
| Frame + page table | The memory slot and tracking metadata for a resident page. |
| Tuple / record | The actual stored row/record represented inside a page. |
| Query execution | Consumes data through memory-resident pages and performs computation on records. |

**Research relevance.** This physical perspective is relevant because vector search is not only a mathematical nearest-neighbor problem. A real index must be stored, loaded, traversed, and searched efficiently on hardware. Therefore, page layout, memory residency, locality, cache behavior, and the cost of accessing many candidate nodes are likely to matter when studying a CPU-oriented vector-search system.

## 4. Initial Investigation of the Thesis Topic (Not Yet Deeply Learned)

I have also investigated the main concepts and research resources that I need to learn next. These are preliminary working definitions, not claims of full mastery.

| Topic | Current basic understanding | Status |
|---|---|---|
| Vector database | A database/search system that stores vector embeddings and supports similarity search, often together with structured metadata. | Investigated; not yet studied deeply |
| Embedding / vector representation | A numeric representation of an item in a high-dimensional space where distance/similarity can capture some notion of relatedness. | Basic concept understood |
| Nearest Neighbor Search | Given a query vector, find database vectors with the smallest distance or highest similarity. | Basic concept understood |

| Topic | Current basic understanding | Status |
|---|---|---|
| Approximate Nearest Neighbor (ANN) | Uses an index or search strategy to obtain high-quality nearest neighbors with much less work than exhaustive comparison with every vector. | Investigated; algorithm details next |
| Proximity graph | Represents vectors as graph nodes connected to selected nearby nodes so search can navigate the space instead of scanning everything. | Basic concept understood |
| HNSW | A hierarchical proximity-graph index used for approximate vector search; search navigates graph layers toward promising neighbors. | Investigated; implementation not yet learned |
| Filtered Vector Search / FANNS | Nearest-neighbor search constrained by metadata predicates such as brand, price, category, or multiple attributes. | Core thesis topic; currently learning |
| Filter selectivity | Roughly, the fraction of records satisfying a predicate. Low selectivity means a small valid subset. | Basic concept understood |
| Query-filter correlation | The spatial relationship between where valid metadata occurs and where vectors lie around a query. | Investigated; needs deeper study |
| Recall@K / latency / QPS | Recall measures how many true nearest filtered neighbors are recovered; latency measures response time; QPS measures throughput. | Basic evaluation concepts understood |

### 4.1 Why Filtered Vector Search Is the Chosen Direction

**Initial motivation:** ordinary ANN asks “which vectors are closest?”, while practical systems often ask “which vectors are closest among the records satisfying these metadata conditions?”. This combination makes the search problem more than a pure distance-computation problem.

- Example: retrieve the 10 most similar product vectors subject to brand = Nike AND price < $50.
- A proximity graph can contain many nodes that are close in vector space but fail the metadata predicate.
- The resulting interaction between graph navigation, filtering, and search effort is the central technical area I intend to study.

### 4.2 Initial Research Literature Map

I have identified the following papers as the main literature path for the next stage of study:

- HNSW (Malkov & Yashunin, 2018) — fundamental graph-based ANN index.
- DiskANN (Subramanya et al., 2019) — large-scale graph-based ANN search and the Vamana family of ideas.
- Filtered-DiskANN (WWW 2023) — filtered ANN using filter-aware graph constructions.
- ACORN (SIGMOD 2024) — predicate-aware search over vector embeddings and structured data.
- NaviX (PVLDB 2025) — native vector indexing with predicate-agnostic search behavior and local-selectivity-related search decisions.
- FANNS Unified Benchmark (2025) — benchmark and experimental methodology for filtered ANN.
- JAG, RACORN-1, and BOA (2026) — recent work showing that the area is active and that adaptive, attribute-aware, and low-selectivity techniques are already being explored.

**Important positioning:** I am not yet claiming a novel algorithm. The current research plan is to understand these methods, reproduce representative baselines, identify a concrete and reproducible inefficiency, and then define the exact thesis contribution from evidence.

## 5. What I Can Confidently Report to My Supervisor Now

- I have started the thesis preparation from database-system fundamentals rather than jumping directly into advanced vector-search algorithms.
- I understand the basic physical architecture of a database: persistent files → pages → buffer pool/frames → tuples/records → query processing.
- I understand why memory management and page movement can affect database performance.
- I have selected filtered vector search on proximity graphs as the main research direction and have identified HNSW, ANN, filtering/selectivity, and recent filtered-search papers as the next learning stage.
- I have not yet implemented ACORN/HNSW or reproduced research benchmarks; these are planned next rather than being presented as completed work.
- The exact optimization contribution will be finalized after literature review and baseline experiments.

## 6. Next Learning & Research Plan

| Stage | Main topics/resources | Goal |
|---|---|---|
| A | CMU 15-445 #05 Database Storage II; #08 Indexes & Filters I; #09 Vector Indexes + Inverted Indexes + Skip Lists + Bloom Filters | Build the database-index foundation directly connected to vector search. |
| B | CMU Database Group: Qdrant Vector Search Engine Internals (2023) | Understand how a real vector search engine represents vectors, builds indexes, and handles filtering. |
| C | HNSW paper + a small implementation/read-through | Understand graph construction, search traversal, key parameters, and complexity/performance trade-offs. |
| D | Filtered-DiskANN, ACORN, NaviX, FANNS benchmark, JAG, RACORN-1, BOA | Map existing solutions, assumptions, datasets, metrics, and limitations. |
| E | Implement/reproduce one or two baselines in C++ | Move from conceptual knowledge to measurable experimental understanding. |
| F | Profiling and controlled experiments | Identify one reproducible bottleneck that can become the thesis research question. |

## 7. Sources Consulted / Learning Resources

[1] CMU 15-445/645 Intro to Database Systems (Fall 2025), official schedule. Lectures #01, #03, #04, and #08–#10. https://15445.courses.cs.cmu.edu/fall2025/schedule.html
[2] Lecture #03: “Database Storage: Files, Pages, Tuples” (CMU Database Group, 2025). YouTube ID: PRLXdIMJhOg.
[3] Lecture #04: “Memory Management & Buffer Pools” (CMU Database Group, 2025). YouTube ID: 8-2yv4z0VZc.
[4] Vasnetsov, A. (2023). “Qdrant - Vector Search Engine Internals.” CMU Database Group seminar. https://www.youtube.com/watch?v=bU38Ovdh3NY
[5] Malkov, Y. A., & Yashunin, D. A. (2018). Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. IEEE TPAMI.
[6] Subramanya, S. V., et al. (2019). DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. NeurIPS.
[7] Gollapudi, S., et al. (2023). Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. WWW 2023.
[8] Patel, L., Kraft, P., Guestrin, C., & Zaharia, M. (2024). ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. SIGMOD / PACMMOD.
[9] Sehgal, G., & Salihoğlu, S. (2025). NaviX: A Native Vector Index Design for Graph DBMSs With Robust Predicate-Agnostic Search Performance. PVLDB 18(11).
[10] Shi, J., Cai, Y., & Zheng, W. (2025). Filtered Approximate Nearest Neighbor Search: A Unified Benchmark and Systematic Experimental Study. arXiv:2509.07789.
[11] Xu, H., et al. (2026). JAG: Joint Attribute Graphs for Filtered Nearest Neighbor Search. arXiv:2602.10258.
[12] Kim, Y., & Choe, G. (2026). RACORN-1: Adaptive Recall-Preserving Speedup for Low-Selectivity Filtered Vector Search. arXiv:2607.00768.
[13] Tumpa, F. A., & Gupta, R. (2026). BOA: Beamwidth Online Adaptation for Filtered-ANNS on a GPU. arXiv:2609.16175.

**Current thesis stage:** Foundation and literature-orientation stage. The next milestone is not “invent an algorithm,” but “reach a reproducible baseline and understand the failure/performance regime well enough to define a precise research question.”